



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 5

Grupo 3

Tecnologías de Datos Masivos /
Big Data Technology

Table of Contents

Introducción	3
Contexto	3
Descripción del problema y objetivos del sprint.....	3
Justificación	3
Metodología	5
Entorno de Desarrollo	5
Pruebas realizadas	5
Resultados	6
Conclusión	7
Anexo	8

Introducción

Contexto

El mercado de criptomonedas genera de forma continua grandes volúmenes de datos de cotización en tiempo real. La capacidad de capturar, transmitir y procesar estos datos de manera inmediata resulta fundamental para construir sistemas de análisis financiero capaces de reaccionar a las condiciones del mercado sin demora.

En este contexto, las arquitecturas de streaming basadas en sistemas de mensajería distribuida como Apache Kafka y motores de procesamiento en tiempo real como Spark Structured Streaming se han consolidado como el estándar de facto en entornos de Big Data para el tratamiento de flujos de eventos de alta frecuencia.

Este sprint extiende la arquitectura construida en iteraciones anteriores incorporando una capa de ingesta y análisis en tiempo real, que cierra el ciclo de procesamiento desde la fuente de datos bruta hasta la producción de un indicador financiero calculado de forma continua.

Descripción del problema y objetivos del sprint

El objetivo de este sprint es implementar un pipeline de datos en tiempo real completo, de extremo a extremo, que cubra dos etapas diferenciadas pero estrechamente relacionadas:

- **Ingesta en tiempo real:** captura de los datos de cotización del par BTCUSDT desde la API de Binance mediante un productor Kafka que publica un mensaje al cierre de cada vela de 1 minuto, incluyendo símbolo, timestamp en UTC, precio de cierre y volumen.
- **Análisis en streaming:** consumo de los mensajes publicados en Kafka y cálculo continuo del indicador VWAP (Volume Weighted Average Price) por ventanas temporales de 5 minutos, publicando el resultado en un topic de salida.

Ambas etapas conforman un flujo de datos cohesionado: Binance WebSocket → productor Kafka → topic de entrada → Spark Structured Streaming → cálculo VWAP → topic de salida.

Justificación

El indicador VWAP es ampliamente utilizado en el análisis técnico de mercados financieros como referencia del precio justo de un activo durante un período determinado. Al ponderar el precio de cierre por el volumen negociado en cada intervalo, el VWAP reduce el impacto de operaciones de poco volumen que podrían distorsionar el precio medio simple.

La elección de Apache Kafka como bus de mensajes garantiza la durabilidad, el orden y la escalabilidad en la transmisión de eventos entre el productor y el motor de análisis. Por su parte, Spark Structured Streaming proporciona las primitivas necesarias para definir ventanas temporales deslizantes, aplicar watermarks que gestionen la llegada tardía de datos y publicar los resultados de forma continua.

La combinación de ambas tecnologías permite construir un sistema de análisis financiero en tiempo real robusto, escalable y alineado con las arquitecturas de referencia utilizadas en la industria del Big Data.

Metodología

Entorno de Desarrollo

El pipeline se ha implementado y ejecutado sobre el clúster del entorno de prácticas del ICAI. El productor se ha desarrollado en Python instalando previamente la librería python-binance mediante pip3, mientras que el job de streaming se ha implementado con PySpark y se despliega a través de spark-submit.

Las tecnologías y componentes utilizados son:

- **Apache Kafka** (broker: 192.168.80.34:9092) como sistema de mensajería distribuida para la transmisión de eventos entre el productor y el consumidor.
- **python-binance / ThreadedWebsocketManager** para la suscripción al stream kline_1m de Binance y la detección del cierre de cada vela.
- **Apache Spark Structured Streaming** para el procesamiento en tiempo real, el cálculo del VWAP por ventanas de 5 minutos y la publicación del resultado.
- **Formato JSON** para la serialización y deserialización de los mensajes Kafka en ambos extremos del pipeline.

Pruebas realizadas

Las pruebas se realizaron ejecutando ambos componentes del pipeline de forma simultánea sobre el clúster del entorno de prácticas. En primer lugar se lanzó el script del productor, comprobando que publicaba mensajes en el topic gittba_BTC únicamente al cierre de cada vela de 1 minuto (campo "x": true en la respuesta de Binance), con el timestamp en milisegundos y el campo @timestamp en formato UTC ISO 8601 sin zona horaria.

Con el productor activo, se lanzó el job de Spark Structured Streaming mediante spark-submit. Se verificó que Spark procesaba correctamente los micro-batches, cerraba las ventanas temporales de 5 minutos al recibir datos suficientes y publicaba el VWAP calculado en el topic de salida gittba_BTC_VWAP con los campos window_start, window_end, symbol y vwap.

```
(venv) gonzaloalonsolidon@Gonzalos-MacBook-Pro trading-view-big-data % python pipelines/spark/binance_real_time_btc.py
/Users/gonzaloalonsolidon/Desktop/Repos/Cursor/trading-view-big-data/venv/lib/python3.9/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning:
urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/
issues/3020
  warnings.warn(
Escuchando Binance y publicando en Kafka. Pulsa ENTER para salir.
Publicado en gittba_BTC | key=BTCUSDOT | ts_ms=1774473539999 | value={'symbol': 'BTCUSDOT', '@timestamp': '2026-03-25T21:18:59Z', 'close': 71087.3
2, 'volume': 6.02751}
Publicado en gittba_BTC | key=BTCUSDOT | ts_ms=1774473599999 | value={'symbol': 'BTCUSDOT', '@timestamp': '2026-03-25T21:19:59Z', 'close': 71082.6
7, 'volume': 0.93972}
□
```

Figure 1: Productor

```
(venv) gonzaloalonsolidon@Gonzalos-MacBook-Pro trading-view-big-data % rm -rf /tmp/vwap_btc_checkpoint

spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.2 pipelines/spark/spark_streaming_vwap_btc.py

mnw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3/userFiles-e636d416-7ef4-4d15-818f-82e307f13d95/org.slf4j_slf4j-api-2.0.16.jar
26/03/25 22:23:26 INFO Executor: Adding file:/private/var/folders/hw/k9dl2bvxx033g270sxxkmnw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3
/userFiles-e636d416-7ef4-4d15-818f-82e307f13d95/org.slf4j_slf4j-api-2.0.16.jar to class loader default
26/03/25 22:23:26 INFO Executor: Fetching spark://192.168.1.184:50681/jars/org.apache.spark_spark-token-provider-kafka-0-10_2.13-4.0.2.jar with
timestamp 1774473805663
26/03/25 22:23:26 INFO Utils: Fetching spark://192.168.1.184:50681/jars/org.apache.spark_spark-token-provider-kafka-0-10_2.13-4.0.2.jar to /priv
ate/var/folders/hw/k9dl2bvxx033g270sxxkmnw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3/userFiles-e636d416-7ef4-4d15-818f-82e307f13d95/fe
tchFileTemp9689411317751850138.tmp
26/03/25 22:23:26 INFO Utils: /private/var/folders/hw/k9dl2bvxx033g270sxxkmnw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3/userFiles-e636
d416-7ef4-4d15-818f-82e307f13d95/fetchFileTemp9689411317751850138.tmp has been previously copied to /private/var/folders/hw/k9dl2bvxx033g270sxxkm
nw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3/userFiles-e636d416-7ef4-4d15-818f-82e307f13d95/org.apache.spark_spark-token-provider-kaf
ka-0-10_2.13-4.0.2.jar
26/03/25 22:23:26 INFO Executor: Adding file:/private/var/folders/hw/k9dl2bvxx033g270sxxkmnw0000gn/T/spark-029959db-b3bf-4baf-b24e-2caaf1a02da3
/userFiles-e636d416-7ef4-4d15-818f-82e307f13d95/org.apache.spark_spark-token-provider-kafka-0-10_2.13-4.0.2.jar to class loader default
26/03/25 22:23:26 INFO NettyBlockTransferService: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 50684.
26/03/25 22:23:26 INFO NettyBlockTransferService: Server created on 192.168.1.184:50684
26/03/25 22:23:26 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/03/25 22:23:26 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 192.168.1.184, 50684, None)
26/03/25 22:23:26 INFO BlockManagerMasterEndpoint: Registering block manager 192.168.1.184:50684 with 434.4 MiB RAM, BlockManagerId(driver, 192.
168.1.184, 50684, None)
26/03/25 22:23:26 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 192.168.1.184, 50684, None)
26/03/25 22:23:26 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 192.168.1.184, 50684, None)
26/03/25 22:23:28 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
```

Figure 2: Submittal de Kafka

Resultados

El pipeline completo funcionó correctamente de extremo a extremo. Los mensajes publicados en el topic gittba_BTC presentaron el formato JSON esperado, con el timestamp obtenido directamente de Binance y el campo @timestamp correctamente formateado en UTC ISO 8601, cumpliendo las especificaciones de la historia de usuario.

El job de Spark Structured Streaming calculó el indicador VWAP para cada ventana temporal de 5 minutos y publicó los resultados en el topic gittba_BTC_VWAP. Los mensajes de salida incluyeron correctamente los límites temporales de cada ventana (window_start y window_end), el símbolo y el valor del VWAP calculado, tal y como especifica la historia de usuario.

```
(venv) gonzaloalonsolidon@Gonzalos-MacBook-Pro trading-view-big-data % python pipelines/spark/vwap_consumer.py

Escuchando topic gittba_BTC_VWAP...

offset=8 | key=BTCUSD | window_start=2026-03-25 22:20:00 | window_end=2026-03-25 22:25:00 | symbol=BTCUSD | vwap=71017.7094034416
```

Figure 3: VWAP recibido

Conclusión

En este sprint se ha implementado un pipeline de datos en tiempo real de extremo a extremo que cubre desde la captura de cotizaciones hasta la producción de un indicador financiero calculado de forma continua. El productor Kafka desarrollado se suscribe al stream de velas de 1 minuto de Binance y publica cada evento de cierre en el topic correspondiente, con el timestamp en formato UTC obtenido directamente de la fuente de datos, lo que garantiza la trazabilidad temporal de todos los mensajes.

El job de Spark Structured Streaming consume ese flujo de mensajes y calcula el indicador VWAP por ventanas de 5 minutos. La aplicación de un watermark permite gestionar la llegada tardía de eventos y liberar el estado de las ventanas ya cerradas, asegurando la corrección del cálculo incluso ante fluctuaciones en la latencia de la red. Los resultados se publican en un topic de salida con el formato JSON especificado en la historia de usuario.

En conjunto, el trabajo demuestra la capacidad de Apache Kafka y Apache Spark Structured Streaming para construir pipelines de análisis en tiempo real robustos y escalables sobre datos financieros de alta frecuencia, sentando las bases para la incorporación de indicadores adicionales en iteraciones futuras.

Anexo

Enlace a GitHub: <https://github.com/GonAlonsoLid/trading-view-big-data>